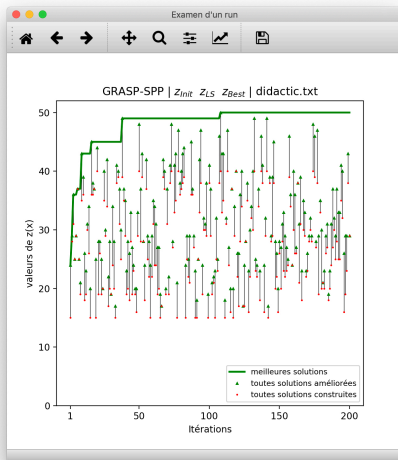


Selected topics

Plotting



Overview

- ▶ **UnicodePlots**: a quick and easy way to draw plots in the REPL

<https://github.com/Evizero/UnicodePlots.jl>

- ▶ **PyPlot**: a collection of command style functions that make matplotlib work like MATLAB

<https://github.com/JuliaPy/PyPlot.jl>

<https://matplotlib.org/tutorials/introductory/pyplot.html>

- ▶ **Plots**: a high-level plotting package that interfaces with other plotting packages

<https://github.com/JuliaPlots/Plots.jl>

<http://docs.juliaplots.org>

To avoid any interactions (and thus error messages) between plotting packages, use only one plotting package simultaneously.

Overview

- ▶ **UnicodePlots**: a quick and easy way to draw plots in the REPL

<https://github.com/Evizero/UnicodePlots.jl>

- ▶ **PyPlot**: a collection of command style functions that make matplotlib work like MATLAB

<https://github.com/JuliaPy/PyPlot.jl>

<https://matplotlib.org/tutorials/introductory/pyplot.html>

- ▶ **Plots**: a high-level plotting package that interfaces with other plotting packages

<https://github.com/JuliaPlots/Plots.jl>

<http://docs.juliaplots.org>

To avoid any interactions (and thus error messages) between plotting packages, use only one plotting package simultaneously.

Overview

- ▶ **UnicodePlots**: a quick and easy way to draw plots in the REPL

<https://github.com/Evizero/UnicodePlots.jl>

- ▶ **PyPlot**: a collection of command style functions that make matplotlib work like MATLAB

<https://github.com/JuliaPy/PyPlot.jl>

<https://matplotlib.org/tutorials/introductory/pyplot.html>

- ▶ **Plots**: a high-level plotting package that interfaces with other plotting packages

<https://github.com/JuliaPlots/Plots.jl>

<http://docs.juliaplots.org>

To avoid any interactions (and thus error messages) between plotting packages, use only one plotting package simultaneously.

UnicodePlots

Install:

```
using Pkg  
Pkg.add("UnicodePlots")
```

Setup:

```
using UnicodePlots
```

Example:

```
julia> f(x)=4*x+10  
julia> X=[x for x=-5:5]; Y=[f(x) for x in X]
```

```
julia> lineplot(X,Y)
```

```
julia> lineplot(X,Y,title="y=f(x)",name="y=4x+10")
```

UnicodePlots

Install:

```
using Pkg  
Pkg.add("UnicodePlots")
```

Setup:

```
using UnicodePlots
```

Example:

```
julia> f(x)=4*x+10  
julia> X=[x for x=-5:5]; Y=[f(x) for x in X]
```

```
julia> lineplot(X,Y)
```

```
julia> lineplot(X,Y,title="y=f(x)",name="y=4x+10")
```

UnicodePlots

Install:

```
using Pkg  
Pkg.add("UnicodePlots")
```

Setup:

```
using UnicodePlots
```

Example:

```
julia> f(x)=4*x+10  
julia> X=[x for x=-5:5]; Y=[f(x) for x in X]
```

```
julia> lineplot(X,Y)
```

```
julia> lineplot(X,Y,title="y=f(x)",name="y=4x+10")
```

Install:

```
using Pkg  
Pkg.add("PyPlot")
```

Setup:

```
using PyPlot
```

Example:

```
julia> f(x)=4*x+10  
julia> X=[x for x=-5:5]; Y=[f(x) for x in X]
```

```
julia> plot(X,Y)
```


Install:

```
using Pkg  
Pkg.add("PyPlot")
```

Setup:

```
using PyPlot
```

Example:

```
julia> f(x)=4*x+10  
julia> X=[x for x=-5:5]; Y=[f(x) for x in X]
```

```
julia> plot(X,Y)
```

Example (Cont'd):

```
julia> title("y=f(x)")
julia> xlabel("x values"); ylabel("y values")
julia> grid()
julia> plot(X,Y,label="y=4x+10")
julia> legend(loc=4, fontsize="small")
julia> xlim(-10,10); ylim(-20,40)
```

```
julia> plot(X,Y,label="y=4x+10",
            color="red",linewidth=2.0,linestyle="--")
```

```
julia> figure("Example with PyPlot",figsize=(6,6))
```

Example (Cont'd):

```
julia> title("y=f(x)")
julia> xlabel("x values"); ylabel("y values")
julia> grid()
julia> plot(X,Y,label="y=4x+10")
julia> legend(loc=4, fontsize="small")
julia> xlim(-10,10); ylim(-20,40)
```

```
julia> plot(X,Y,label="y=4x+10",
            color="red",linewidth=2.0,linestyle="--")
```

```
julia> figure("Example with PyPlot",figsize=(6,6))
```

Example (Cont'd):

```
julia> title("y=f(x)")
julia> xlabel("x values"); ylabel("y values")
julia> grid()
julia> plot(X,Y,label="y=4x+10")
julia> legend(loc=4, fontsize="small")
julia> xlim(-10,10); ylim(-20,40)
```

```
julia> plot(X,Y,label="y=4x+10",
            color="red",linewidth=2.0,linestyle="--")
```

```
julia> figure("Example with PyPlot",figsize=(6,6))
```

PyPlot: example of a 2x2 subplot

```
julia> fig = figure("pyplot subplot",figsize=(5,5))
julia> subtitle("Example of a 2x2 Subplot")
```

```
julia> subplot(221) #-----
julia> PyPlot.title("221 - NW"); grid("on")
julia> plot(X,Y,color="red")
```

```
julia> subplot(222) #-----
julia> PyPlot.title("222 - NE")
julia> scatter(X,Y)
julia> ax = subplot(223) #-----
julia> PyPlot.title("223 - SW")
julia> x = randn(100); nb = 25
julia> hist(x,nb)
julia> subplot(224) #-----
julia> PyPlot.title("224 - SE"); xlabel("X axis")
julia> Xc=[i for i=0:0.1:π]; Yc = sin.(Xc)
```

PyPlot: example of a 2x2 subplot

```
julia> fig = figure("pyplot subplot",figsize=(5,5))
julia> suptitle("Example of a 2x2 Subplot")
```

```
julia> subplot(221)                                #-----
julia> PyPlot.title("221 - NW"); grid("on")
julia> plot(X,Y,color="red")
```

```
julia> subplot(222)                                #-----
julia> PyPlot.title("222 - NE")
julia> scatter(X,Y)
julia> ax = subplot(223)                            #-----
julia> PyPlot.title("223 - SW")
julia> x = randn(100); nb = 25
julia> hist(x,nb)
julia> subplot(224)                                #-----
julia> PyPlot.title("224 - SE"); xlabel("X axis")
julia> Xc=[i for i=0:0.1:2π]; Yc = sin.(Xc)
```

PyPlot: example of a 2x2 subplot

```
julia> fig = figure("pyplot subplot",figsize=(5,5))
julia> suptitle("Example of a 2x2 Subplot")
```

```
julia> subplot(221) #-----
julia> PyPlot.title("221 - NW"); grid("on")
julia> plot(X,Y,color="red")
```

```
julia> subplot(222) #-----
julia> PyPlot.title("222 - NE")
julia> scatter(X,Y)
julia> ax = subplot(223) #-----
julia> PyPlot.title("223 - SW")
julia> x = randn(100); nb = 25
julia> hist(x,nb)
julia> subplot(224) #-----
julia> PyPlot.title("224 - SE"); xlabel("X axis")
julia> Xc=[i for i=0:0.1:1]; Yc = sin(Xc)
```

Plots (1/2)



Plots supports multiple backends:

- ▶ UnicodePlots
- ▶ PyPlot
- ▶ GR
- ▶ Plotly(JS)
- ▶ etc.

within the REPL
the universe of Matplotlib
simple and fast
interactive

Without changing syntax, we can create plots with different backends!

Install:

```
using Pkg  
Pkg.add("Plots")
```

Setup:

```
using Plots
```


Plots (1/2)



Plots supports multiple backends:

- ▶ UnicodePlots
- ▶ PyPlot
- ▶ GR
- ▶ Plotly(JS)
- ▶ etc.

within the REPL
the universe of Matplotlib
simple and fast
interactive

Without changing syntax, we can create plots with different backends!

Install:

```
using Pkg  
Pkg.add("Plots")
```

Setup:

```
using Plots
```

Plots (2/2)



Example with GR:

```
julia> gr()  
julia> f(x)=4*x+10  
julia> X=[x for x=-5:5]; Y=[f(x) for x in X]  
julia> plot(X,Y, label="line")
```

```
julia> scatter!(X,Y, label="points")
```

```
julia> title!("y=f(x)")  
julia> xlabel!("x values"); ylabel!("y values")
```

Example with UnicodePlots:

```
julia> unicodeplots()  
julia> plot(X,Y, label="line")
```



Example with GR:

```
julia> gr()  
julia> f(x)=4*x+10  
julia> X=[x for x=-5:5]; Y=[f(x) for x in X]  
julia> plot(X,Y, label="line")
```

```
julia> scatter!(X,Y, label="points")
```

```
julia> title!("y=f(x)")  
julia> xlabel!("x values"); ylabel!("y values")
```

Example with UnicodePlots:

```
julia> unicodeplots()  
julia> plot(X,Y, label="line")
```

Plots (2/2)



Example with GR:

```
julia> gr()  
julia> f(x)=4*x+10  
julia> X=[x for x=-5:5]; Y=[f(x) for x in X]  
julia> plot(X,Y, label="line")
```

```
julia> scatter!(X,Y, label="points")
```

```
julia> title!("y=f(x)")  
julia> xlabel!("x values"); ylabel!("y values")
```

Example with UnicodePlots:

```
julia> unicodeplots()  
julia> plot(X,Y, label="line")
```



Example with GR:

```
julia> gr()  
julia> f(x)=4*x+10  
julia> X=[x for x=-5:5]; Y=[f(x) for x in X]  
julia> plot(X,Y, label="line")
```

```
julia> scatter!(X,Y, label="points")
```

```
julia> title!("y=f(x)")  
julia> xlabel!("x values"); ylabel!("y values")
```

Example with UnicodePlots:

```
julia> unicodeplots()  
julia> plot(X,Y, label="line")
```

